OPTIMISATION DE MODELES NEURONAUX A OSCILLATEURS COUPLÉS - LIVRABLE 4 : ANALYSE FONCTIONNELLE

Da Silva Tristan - Battaglia Charles-Antoine
Botté-Magalhaes Jules - Gauthier Keanu - Fraysse Ianis

JUNIA - ISEN - AP4

`https://github.com/CharlesBta/Modele-Neuronaux-RD.git`

Lexique

Analyse fonctionnelle	Démarche qui décrit les fonctions qu'un système doit remplir, indépendamment des solutions techniques qui les réalisent.
Bête à cornes	Diagramme d'expression du besoin : à qui le système rend-il service, sur quoi agit-il, dans quel but ?
Caractérisation	Définition mesurable d'une fonction : critère d'appréciation, niveau visé, tolérance admise et flexibilité.
Connectome	Matrice $N \times N$ décrivant les connexions synaptiques entre les N neurones du réseau.
CSR	*Compressed Sparse Row* : stockage d'une matrice creuse par lignes compactées, adapté au connectome sur **GPU**.
EDO	Équation différentielle ordinaire ; le modèle de **Hodgkin-Huxley** est un système d'**EDO**.
Flexibilité	Niveau de négociabilité d'un critère, de 0 (aucune marge) à 4 (critère indicatif).
Fonction principale	Fonction qui justifie l'existence du système face à son client.
Fonction de service	Fonction qui rend service à l'utilisateur ou s'adapte à son environnement.
Fonction de contrainte	Fonction imposée par le client, le matériel ou le calendrier, non négociable dans son principe.
Pieuvre	Diagramme des interactions entre le système et les éléments de son environnement.
Speedup	Rapport des temps de calcul entre la référence **CPU** et la version **GPU** pour une même simulation.
Tri croisé	Comparaison des fonctions deux à deux pour les hiérarchiser et en déduire des priorités.
VRAM	Mémoire embarquée du **GPU** ; sur la **RTX 5090**, elle borne la taille des connectomes simulables.

Table des matières

Lexique **1**

1 Introduction **3**
1.1 Objet du document . . . 3
1.2 Rappel du défi technique issu du benchmarking . . . 3

2 Expression du besoin **4**

3 Graphe des fonctions **5**
3.1 Pieuvre du système . . . 5
3.2 Tableau des fonctions . . . 5

4 Caractérisation des fonctions **7**

5 Hiérarchisation des fonctions **9**
5.1 Méthode du tri croisé . . . 9
5.2 Matrice du tri croisé . . . 9
5.3 Classement des fonctions . . . 9
5.4 Fonctions retenues pour le POC . . . 10

6 Conclusion et perspectives **11**

A Sources du tableau de caractérisation **12**

Bibliographie **13**

Introduction

1.1 Objet du document

Ce document est le livrable L4 du projet R&D : l'**analyse fonctionnelle**. La note de clarification (L2) a posé le besoin du laboratoire et le périmètre ; l'état de l'art (L3) a montré ce que la littérature mesure déjà sur l'accélération **GPU** de simulations de neurones. Avant de choisir des solutions techniques, il faut décrire ce que le système *doit faire* pour son utilisateur, sans préjuger du *comment*.

L'analyse fonctionnelle sert ici trois objectifs :

— **formaliser les fonctions** que l'environnement de simulation doit remplir, en les classant en fonctions principales (**FP**), fonctions de service (**FS**) et fonctions de contrainte (**FC**) ;

— **caractériser chaque fonction** par un critère mesurable, un niveau visé, une tolérance et une flexibilité, pour que la suite du projet dispose d'exigences vérifiables ;

— **hiérarchiser les fonctions** par la méthode du tri croisé, afin de délimiter ce que le POC réalisera en priorité dans la limite des 100 h du second semestre.

Ce document s'appuie sur le besoin exprimé en L2, sur le socle de code fourni (modèle de **Hodgkin-Huxley**, synapses, solveur **CPU** de référence) et sur les conclusions du benchmark L3. Il prépare le livrable L5 (FAST et schéma synoptique) : les fonctions prioritaires dégagées ici seront déclinées en solutions techniques dans le FAST.

1.2 Rappel du défi technique issu du benchmarking

L'état de l'art a comparé cinq approches d'accélération **GPU** de simulations neuronales : **CUDA** [1], le générateur de code GeNN [2], le backend Brian2CUDA [3], un solveur d'**EDO** multi-**GPU** en **Julia** [4] et le simulateur multi-**GPU** SimHH [5].

Trois enseignements en ressortent :

— **le gain est réel** : les outils publiés annoncent des accélérations de $\times 100$ à $\times 1000$ face à une référence **CPU** mono-cœur, jusqu'à $\times 150$ pour SimHH, le plus proche de notre sujet (réseaux de **Hodgkin-Huxley**) ;

— **aucun ne répond directement au besoin** : SimHH exige un cluster en **MPI**, GeNN et Brian2CUDA visent des réseaux à impulsions plutôt que notre couplage oscillateurs synaptiques, et le solveur **Julia** impose de traduire le socle existant ;

— **le verrou est identifié** : le pas de temps s'enchaîne séquentiellement et le connectome, matrice creuse irrégulière, provoque des accès mémoire non uniformes qui plafonnent le parallélisme [6].

Le défi technique que doit relever le POC est donc le suivant : *réussir là où les outils généraux sont trop lourds, un portage **CUDA** ciblé du socle fourni, qui accélère significativement la simulation de réseaux de **Hodgkin-Huxley** couplés sans dérive numérique par rapport à la référence **CPU**, sur une seule **RTX 5090***. Les fonctions décrites dans la suite découlent de ce défi.

Expression du besoin

Le besoin du laboratoire, formulé et validé en L2, rappelé ici car l'analyse fonctionnelle en découle :

> *Le laboratoire a besoin de pouvoir simuler des réseaux de neurones biophysiques de grandes tailles dans un temps de calcul acceptable, afin d'étudier des connectomes assez grands pour être utiles à la recherche, et de savoir si l'architecture **GPU** dont il dispose apporte un gain réel face à la référence **CPU**.*

La bête à cornes ci-dessous, reprise de la note de clarification, formalise ce besoin.

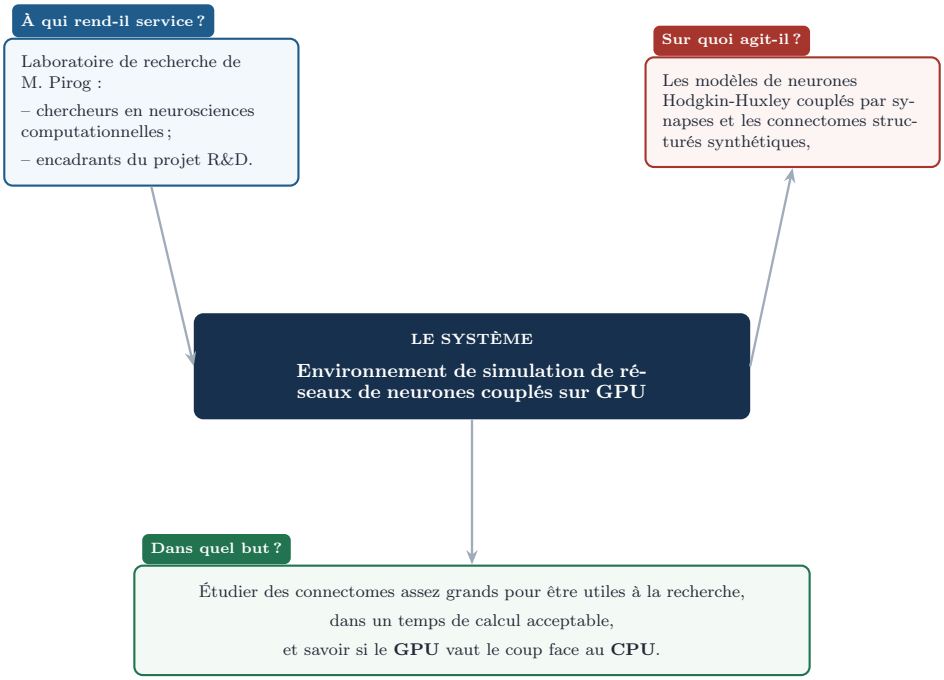


FIGURE 2.1 – Bête à cornes du projet (reprise de la note de clarification)

Graphe des fonctions

3.1 Pieuvre du système

Le système interagit avec les éléments de son environnement : le laboratoire qui l'utilise, le socle de code sur lequel il s'appuie, les connectomes qu'il reçoit en entrée, le serveur **GPU** sur lequel il s'exécute et le calendrier du projet. Le diagramme pieuvre ci-dessous relie le système à ces éléments : une seule bulle au centre pour le produit, et une bulle propre à chaque fonction, placée près du centre et reliée au produit par une seule flèche ; la pastille portée par la flèche donne le code de la fonction, sa couleur indiquant la famille (fonction principale, secondaire ou de contrainte, voir légende).

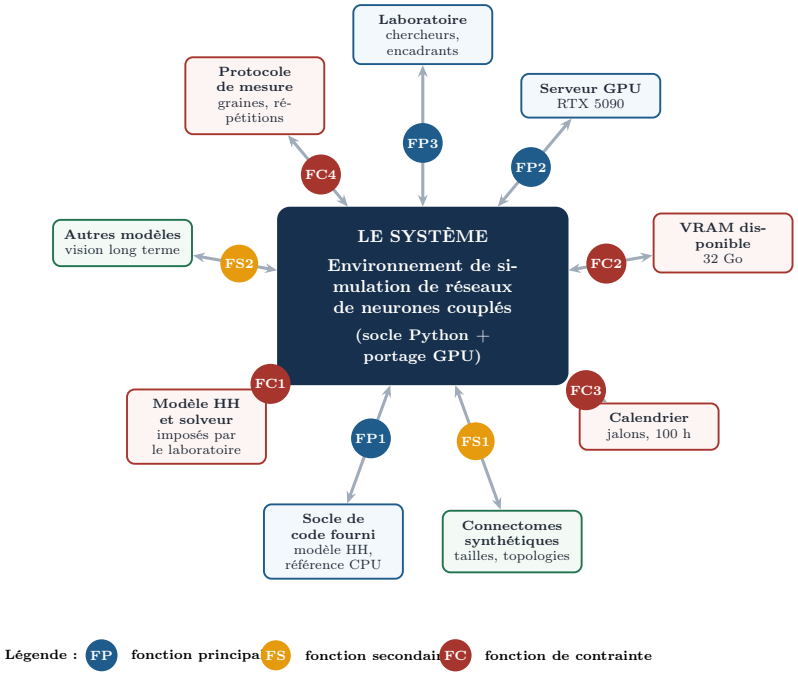


FIGURE 3.1 – Pieuvre du système : interactions avec l'environnement et fonctions associées

3.2 Tableau des fonctions

L'analyse des interactions ci-dessus fait ressortir neuf fonctions, réparties en trois familles. Les fonctions principales rendent le service attendu par le laboratoire ; les fonctions de service adaptent ce service à l'usage ; les fonctions de contrainte sont imposées et bornent la conception.

Code	Fonction	Type
FP1	Simuler des réseaux de neurones **Hodgkin-Huxley** couplés par synapses, en calculant l'évolution des potentiels membranaires à chaque pas de temps.	FP
FP2	Accélérer la simulation sur **GPU** et mesurer le gain de temps de calcul face à la référence **CPU**.	FP

Code	Fonction	Type
FP3	Restituer les résultats aux chercheurs (tracés, temps de calcul, erreurs) sous une forme lisible et exploitable.	FP
FS1	S'adapter à différentes tailles et topologies de connectomes.	FS
FS2	Adapter les équations du modèle neuronal pour simuler d'autres phénomènes (autres modèles de neurones, variantes des équations) sans réécrire la chaîne de simulation.	FS
FC1	Respecter le modèle de **Hodgkin-Huxley** et sa méthode de résolution : mêmes équations, même schéma d'intégration que le socle de référence.	FC
FC2	Fonctionner sur le matériel disponible : une **RTX 5090**, 32 Go de **VRAM**.	FC
FC3	Respecter le temps et le calendrier du projet (32 h d'études au S7, 100 h de réalisation au S8, jalons imposés).	FC
FC4	Garantir des mesures reproductibles : protocole de benchmark documenté, mesures répétées, graines fixées.	FC

Table 3.1: Tableau des fonctions du système

Lecture. FP1 est la fonction qui justifie l'existence du système : sans simulation fidèle du réseau couplé, rien d'autre n'a de sens. FP2 est la raison d'être du projet R&D : le laboratoire veut savoir si le **GPU** apporte un gain réel. FP3 boucle le service : des résultats que seul une machine sait lire ne servent pas des chercheurs. Les fonctions de service préparent l'environnement générique visé par la vision du L2 : FS2 rend les équations du modèle remplaçables pour simuler d'autres phénomènes. Les contraintes bornent le terrain de jeu : le respect du modèle de **Hodgkin-Huxley** et de sa méthode de résolution n'est pas négociable (sinon le speedup ne mesure rien), la **VRAM** et le calendrier fixent les limites physiques et humaines du POC.

Caractérisation des fonctions

Chaque fonction est caractérisée par un critère d'appréciation (ce que l'on mesure), un niveau (la valeur visée), une tolérance (la marge admise) et une flexibilité (de 0 : aucune marge, à 4 : critère indicatif). Les niveaux s'appuient sur le socle fourni, les mesures du benchmark CPU existant et l'état de l'art (les sources sont détaillées en annexe A).

Code	Critère	Description	Niveau	Tolérance	Flex.
FP1	Fidélité de la simulation	Simuler le réseau couplé et reproduire les tracés de référence du solveur CPU	Superposition exacte des tracés de référence sur les réseaux de validation ($N = 2$ à 16)	Erreur relative $\leq 10^{-4}$ par variable d'état	F1
FP2	Gain de temps mesuré	Accélérer la simulation sur GPU et quantifier le speedup face à la référence CPU	Speedup $\geq \times 10$ sur les plus grands réseaux testés	$\pm$ 50 % selon la taille et la topologie	F2
FP3	Restitution des résultats	Fournir aux chercheurs tracés et métriques exploitables en fin de simulation	Tracés des potentiels + tableau CSV/JSON (temps, speedup, erreur) à chaque exécution	Aucune : les sorties font partie du contrat	F0
FS1	Tailles et topologies acceptées	Accepter les connectomes par tailles croissantes et topologies variées	De $N = 2$ (validation) à $N \geq 10\,000$ neurones	Borne haute fonction de la VRAM disponible	F2
FS2	Extensibilité du modèle	Remplacer les équations du modèle neuronal pour simuler d'autres phénomènes, sans réécrire la chaîne de simulation	Équations du modèle isolées dans un module dédié, remplaçable indépendamment du moteur de simulation	Partielle au POC (seul le modèle **HH** est implanté, mais ses équations sont découplées)	F2
FC1	Conformité au modèle	Respecter le modèle de **Hodgkin-Huxley** et sa méthode de résolution, imposés par le socle de référence	Mêmes équations et même schéma de résolution que le socle fourni	Aucune	F0
FC2	Mémoire disponible	Tenir dans la VRAM du GPU, connectome stocké au format creux (CSR)	Connectome CSR résidant en 32 Go de VRAM ; borne physique, non négociable	Aucune	F0
FC3	Temps de travail	Tenir le calendrier : études au S7, 100 h de réalisation au S8, évaluations imposées	POC livré aux jalons du calendrier	Aucune	F0
FC4	Reproductibilité	Donner des mesures que d'autres peuvent refaire et comparer	Chaque mesure répétée au moins 3 fois, graines fixées, protocole documenté	$\pm$ 10 % entre répétitions	F1

Table 4.1: Tableau de caractérisation des fonctions

Justification des flexibilités.

— **FP1 – F1** : la fidélité est le socle de la valeur scientifique du projet ; une erreur relative de 10^{-4} reste toutefois techniquement discutable, d'où une flexibilité faible mais non nulle.

— **FP2 – F2** : le niveau $\times 10$ est prudent au vu de l'état de l'art ($\times 100$ à $\times 1000$ sur des outils génériques) ; le gain réel dépendra fortement de la topologie et des accès mémoire au connectome, d'où une tolérance large et une flexibilité moyenne.

— **FP3 – F0** : des résultats non restitués équivalent à une simulation inutile ; aucune marge.

— **FS1, FS2 – F2** : l'adaptation complète à toutes tailles et l'extensibilité généralisée du modèle relèvent de la vision long terme ; le POC peut se contenter d'un sous-ensemble (chaîne, aléatoire, N borné par la **VRAM**, équations **HH** découplées).

— **FC1 à FC3 – F0** : contraintes imposées par le sujet, le matériel ou le calendrier ; leur violation invalide le projet, pas seulement une performance.

— **FC4 – F1** : trois répétitions suffisent au POC ; le protocole pourra être étoffé si les mesures s'avèrent instables.

Hiérarchisation des fonctions

5.1 Méthode du tri croisé

Les neuf fonctions sont comparées deux à deux : pour chaque paire, la fonction la plus importante reçoit un score.

- **0** : les deux fonctions sont d'importance égale ;
- **1** : la fonction est un peu plus importante que l'autre ;
- **2** : la fonction est plus importante que l'autre ;
- **3** : la fonction est beaucoup plus importante que l'autre.

Les scores de chaque fonction sont additionnés : le total, ramené en pourcentage de la somme générale, hiérarchise les fonctions et délimite le périmètre du POC.

5.2 Matrice du tri croisé

	FP1	**FP2**	**FP3**	**FS1**	**FS2**	**FC1**	**FC2**	**FC3**	**FC4**
FP1	—	FP1 (2)	FP1 (3)	FP1 (3)	FP1 (3)	FP1 (2)	FP1 (3)	FP1 (3)	FP1 (3)
FP2		—	FP2 (1)	FP2 (3)	FP2 (3)	FC1 (2)	FP2 (2)	FP2 (3)	FP2 (3)
FP3			—	FP3 (2)	FP3 (3)	FC1 (2)	FP3 (2)	FP3 (3)	FP3 (3)
FS1				—	FS1 (1)	FC1 (3)	FC2 (2)	FS1 (3)	FS1 (2)
FS2					—	FC1 (3)	FC2 (2)	FS2 (3)	FS2 (2)
FC1						—	FC1 (3)	FC1 (3)	FC1 (3)
FC2							—	FC2 (3)	FC2 (3)
FC3								—	FC4 (2)
FC4									—

Dans chaque case, la fonction retenue comme la plus importante est notée avec son score face à l'autre fonction comparée.

5.3 Classement des fonctions

Les scores sont additionnés pour chaque fonction. Le tableau ci-dessous donne le classement, le poids relatif et le poids cumulé.

Code	Fonction	Points	Poids	Cumul
FP1	Simuler les réseaux **HH** couplés	22	23,9 %	23,9 %
FC1	Respecter le modèle **HH** et sa résolution	19	20,7 %	44,6 %
FP2	Accélérer sur **GPU** et mesurer le speedup	15	16,3 %	60,9 %
FP3	Restituer les résultats	13	14,1 %	75,0 %
FC2	Fonctionner sur le matériel disponible	10	10,9 %	85,9 %
FS1	S'adapter aux tailles et topologies	6	6,5 %	92,4 %
FS2	Adapter les équations du modèle	5	5,4 %	97,8 %
FC4	Garantir des mesures reproductibles	2	2,2 %	100,0 %
FC3	Respecter le calendrier	0	0,0 %	100,0 %
Total		**92**	**100 %**	

Deux constats guident la suite du projet :

— **FP1 domine** le classement : sans simulation du réseau couplé, il n'y a rien à accélérer ni à restituer.

— **FC1 arrive en seconde position**, devant FP2 : un speedup acheté au prix d'un modèle modifié ne vaut rien. Ce résultat tranche une question ouverte depuis le L2 : le respect du modèle prime sur la vitesse. Le projet ne cherche pas à accélérer *à tout prix*, mais à mesurer un gain *sans toucher aux équations ni à la méthode de résolution.*

— **FP2 et FP3 complètent le podium** : le cœur métier du POC est bien la mesure du gain **GPU** et sa restitution au laboratoire.

— **FC3 ferme la marche** : le calendrier ne rend aucun service au laboratoire ; il ne fait qu'encadrer le travail. Il n'en reste pas moins une contrainte dure (flexibilité F0) : un projet hors jalons est un projet non évalué, pas un projet plus léger.

5.4 Fonctions retenues pour le POC

Le POC du second semestre se concentre sur les quatre fonctions de tête, qui pèsent **75,0 %** du poids total : **FP1** (simuler), **FC1** (respect du modèle et de sa résolution), **FP2** (accélérer et mesurer), **FP3** (restituer). Elles forment un périmètre cohérent : la simulation fidèle du socle, son portage **GPU** et le benchmark comparatif documenté.

Les autres fonctions ne disparaissent pas : **FC2** et **FC4** bornent d'ores et déjà la conception (matériel, protocole de mesure) ; **FS1** sera réalisée en partie (tailles croissantes jusqu'à la borne **VRAM**) ; **FS2** sera abordée à minima (équations du modèle isolées dans un module remplaçable). L'environnement générique complet de la vision L2 reste la cible long terme, au-delà du POC.

Conclusion et perspectives

L'analyse fonctionnelle a fait passer le projet du besoin exprimé à neuf fonctions formelles : trois fonctions principales (simuler, accélérer et mesurer, restituer), deux fonctions de service (adaptation aux connectomes, extensibilité du modèle) et quatre contraintes (respect du modèle et de sa résolution, matériel, calendrier, reproductibilité). La caractérisation a fixé pour chacune un critère, un niveau, une tolérance et une flexibilité ; le tri croisé les a hiérarchisées.

Le résultat principal est un périmètre de POC net : **FP1, FC1, FP2 et FP3**, soit 75,0 % du poids. Autrement dit : simuler fidèlement le réseau de **Hodgkin-Huxley** fourni, le porter sur **RTX 5090** en **CUDA**, mesurer le speedup face à la référence **CPU** sans dérive numérique, et rendre les résultats exploitables au laboratoire. La hiérarchie a aussi tranché une question de méthode : le respect du modèle (FC1) pèse plus lourd que la vitesse (FP2), ce qui guidera les arbitrages techniques à venir.

Les prochaines étapes :

1. **décliner les fonctions prioritaires en solutions techniques** dans le livrable L5 (FAST et schéma synoptique) : la FP1 en pipeline de simulation, la FP2 en stratégie de parallélisation par neurone et d'accès au connectome creux ;
2. **vérifier la cohérence des niveaux visés** dès les premiers développements du S8 : le speedup $\geq \times 10$ de FP2 et l'erreur $\leq 10^{-4}$ de FP1 serviront de critères de réussite mesurables ;
3. **maintenir le tableau de caractérisation à jour** : si le protocole de mesure ou le matériel changent, les niveaux et tolérances seront révisés, et la présente note mise à jour en conséquence.

Sources du tableau de caractérisation

Fonctior	Source du niveau et de la tolérance
FP1	Reproduction du socle fourni : le solveur **CPU** de référence est la vérité terrain ; l'erreur relative de 10^{-4} correspond à l'ordre de grandeur des écarts d'intégration acceptés par les solveurs d'**EDO** à pas fixe [1, 2].
FP2	État de l'art L3 : GeNN annonce $\times 200$ face à un cœur **CPU** [3], Brian2CUDA $\times 1000$ face à Brian [4], le solveur **Julia** $\times 20$ à $\times 100$ [5], SimHH $\times 150$ face à un **CPU** mono-fil [6]. Le niveau $\geq \times 10$, prudent, laisse une tolérance de ± 50 % selon la topologie : les accès irréguliers au connectome creux plafonnent le parallélisme [7].
FP3	Choix d'équipe : CSV et JSON sont des formats lisibles partout et convertibles rapidement entre eux ; cohérent avec le benchmark **CPU** existant du socle [1].
FS1	Tailles de validation du benchmark **CPU** existant ($N \in \{2, 4, 8, 16\}$) pour la borne basse ; vision L2 (« plusieurs milliers à dizaines de milliers de neurones ») pour la borne haute ; la **VRAM** de la **RTX 5090** (32 Go) fixe la limite physique [8].
FS2	Vision L2 : environnement générique où les équations du modèle peuvent être remplacées pour simuler d'autres phénomènes. Réduction au POC : équations du modèle **HH** isolées dans un module dédié, remplaçable.
FC1	Socle fourni : le modèle de **Hodgkin-Huxley** et sa méthode de résolution sont imposés par le laboratoire. Note de clarification L2 : « pas d'accélération au prix d'une dérive numérique par rapport au modèle de référence ».
FC2	Fiche technique de la **RTX 5090** : 32 Go de **VRAM** [8] ; stockage CSR du connectome, format standard pour les matrices creuses [7].
FC3	Calendrier du sujet : 32 h encadrées au S7, 100 h de réalisation au S8, évaluations des 08/10 et 22/10.
FC4	Bonne pratique de benchmarking : répéter les mesures pour estimer la variance ; graines fixées pour régénérer les connectomes aléatoires à l'identique.

Références des sources citées :

1. Socle de code fourni (modèle de **Hodgkin-Huxley**, synapses, solveur **CPU** de référence) et benchmark **CPU** existant du dépôt du projet.
2. Benchmark comparatif du livrable L3 (fiches d'analyse des articles).
3. E. Yavuz, J. Turner, T. Nowotny, « GeNN : a code generation framework for accelerated brain simulations », *Scientific Reports*, 2016.
4. D. Alevi et al., « Brian2CUDA : Flexible and Efficient Simulation of Spiking Neural Network Models on GPUs », *Frontiers in Neuroinformatics*, 2022.
5. U. Utkarsh et al., « Automated Translation and Accelerated Solving of Differential Equations on Multiple GPU Platforms », *CMAME*, 2024.
6. M. C. W. Engelen, J.-H. L. F. Betting, C. Strydis, « SimHH : A Versatile, Multi-GPU Simulator for Extended Hodgkin-Huxley Networks », *IEEE Access*, 2025.
7. N. Bell, M. Garland, « Implementing sparse matrix-vector multiplication on throughput-oriented processors », *SC'09*, ACM, 2009.
8. Fiche technique **NVIDIA GeForce RTX 5090**, TechPowerUp.

Bibliographie

[1] J. Nickolls, I. Buck, M. Garland, K. Skadron, « Scalable Parallel Programming with CUDA », *ACM Queue*, vol. 6, n° 2, p. 24–32, mars 2008. DOI : 10.1145/1365490.1365500.

[2] E. Yavuz, J. Turner, T. Nowotny, « GeNN : a code generation framework for accelerated brain simulations », *Scientific Reports*, vol. 6, article 18854, 2016. DOI : 10.1038/srep18854.

[3] D. Alevi, M. Stimberg, H. Sprekeler, K. Obermayer, M. Augustin, « Brian2CUDA : Flexible and Efficient Simulation of Spiking Neural Network Models on GPUs », *Frontiers in Neuroinformatics*, vol. 16, article 883700, 2022. DOI : 10.3389/fninf.2022.883700.

[4] U. Utkarsh, V. Churavy, Y. Ma, T. Besard, et al., « Automated Translation and Accelerated Solving of Differential Equations on Multiple GPU Platforms », *Computer Methods in Applied Mechanics and Engineering*, vol. 419, 2024. DOI : 10.1016/j.cma.2023.116591.

[5] M. C. W. Engelen, J.-H. L. F. Betting, C. Strydis, « SimHH : A Versatile, Multi-GPU Simulator for Extended Hodgkin-Huxley Networks », *IEEE Access*, vol. 13, 2025. DOI : 10.1109/ACCESS.2025.3550444.

[6] N. Bell, M. Garland, « Implementing sparse matrix-vector multiplication on throughput-oriented processors », *Proceedings of the Conference on High Performance Computing, Networking, Storage and Analysis (SC'09)*, ACM, 2009. DOI : 10.1145/1654059.1654078.

[7] « NVIDIA GeForce RTX 5090 », fiche technique, TechPowerUp. www.techpowerup.com/gpu-specs/geforce-rtx-5090.c4216.